Models for Programming Paradigms: Course Syllabus & TOC

- Foundations & Preliminaries
  - **Key Concepts:** Syntax vs Semantics vs Pragmatics
  - **Methods:** Operational Semantics (SOS: Big-Step and Small-Step), Denotational, Axiomatic
  - **Properties:** Termination, Determinacy, Compositionality
- Math & Logic
  - **Induction:** Mathematical, Structural, Well-Founded, and Rule Induction
  - **Domain Theory:** Partial Orders (PO), CPO, Monotone and Continuous Functions
  - **Fixpoint:** Kleene's Theorem ($\text{fix}(f) = \bigsqcup f^n(\bot)$) and the Immediate Consequence Operator (ICO)
- IMP Semantics
  - **IMP Language:** Syntax and States (Σ)
  - **Operational Semantics:** Inference rules for `Aexp`, `Bexp`, `Com`
  - **Denotational Semantics:** Functions $\mathcal{C}[\![c]\!]$, handling *Lifting*, and `while` denotation
  - **Equivalence:** Consistency Theorem between OS and DS
- HOFL
  - **Higher-Order Functional Language:** Syntax, Types, and Inference Rules
  - **Evaluation:** Canonical Forms, Lazy (Call-by-Name) vs Eager (Call-by-Value)
  - **Semantic Domains:** Continuous function spaces, semantics of λ-abstractions and recursion
  - **Haskell:** Conceptual mapping (lists, lazy evaluation)
- Concurrency
  - **Paradigms:** Message Passing (Erlang) vs Shared Memory/Channels (Go)
  - **CCS:** Syntax, LTS, and transition rules (τ)
  - **Bisimulation:** Definition of Strong Bisimulation, Attacker/Defender game
  - **Logic:** HML (Hennessy-Milner Logic) and Characterization Theorem
- Real Languages
  - **Haskell:** Functional patterns (Guards, Data types), Type Classes, Lazy Evaluation implementation
  - **Erlang:** Actor Model, Asynchronous Message Passing, `receive` with timeout
  - **Go:** Goroutines, Buffered vs Unbuffered Channels, `select` non-determinism
- Advanced Concurrency
  - **Weak Bisimulation:** Abstraction from internal actions (τ), Weak Transition ($\overset{\alpha}{\Longrightarrow}$), Observational Congruence
  - **Temporal Logics:** Linear Time (LTL) vs Branching Time (CTL)
  - **Mu-Calculus:** Syntax and Fixpoints for Safety (ν) and Liveness (μ) properties

Foundations & Preliminaries

This module establishes the rigorous mathematical framework necessary to define, analyze, and compare the semantics of programming languages.

1. Structure and Meaning of a Language

A programming language is defined by four essential components:

1. **Syntax:** Defines the sequences of symbols that constitute well-formed programs. Formalized via regular expressions, context-free grammars (CFG), or BNF notation.
2. **Types:** Restrict the syntax to enforce safety properties and prevent runtime errors. Type systems are defined via logical inference rules.
3. **Pragmatics:** Guidelines on the effective use of the language, best practices, and common patterns.
4. **Semantics:** Assigns an unambiguous meaning to well-typed programs, providing a formal model for programmers and implementers.

---

2. Methods of Formal Semantics

There are three main approaches to formalizing the meaning of programs.

2.1 Operational Semantics (OS)

Describes meaning in terms of execution on an abstract machine. It is based on the **SOS (Structural Operational Semantics)** formalism by Gordon Plotkin.

- **Small-step (SOS):** Describes the step-by-step evolution ($e_0 \to e_1 \to \ldots$). Fundamental for modeling concurrency and non-terminating systems.
- **Big-step (Natural Semantics):** Directly relates the initial state to the final result, hiding intermediate steps.
  - *Judgment for Expressions:* $\langle a, \sigma \rangle \to n$ (expression a in state σ evaluates to n).
  - *Judgment for Commands:* $\langle c, \sigma \rangle \to \sigma'$ (command c in state σ converges to state σ').

  Note: Big-step semantics does not produce a derivation tree for programs that diverge (do not terminate).

2.2 Denotational Semantics (DS)

More abstract than OS, it models programs as **mathematical functions** over specific domains, ignoring execution details.

Compositionality Principle: The meaning of a compound construct depends *only* on the meanings of its constituents:

$$\mathcal{M}[\![OP(c_1, c_2)]\!] = F(\mathcal{M}[\![c_1]\!], \mathcal{M}[\![c_2]\!])$$

Interpretation functions for an imperative language (IMP):

- **Aexp:** $\mathcal{A}[\![\cdot]\!] : \text{Aexp} \to (\Sigma \to \mathbb{Z})$
- **Bexp:** $\mathcal{B}[\![\cdot]\!] : \text{Bexp} \to (\Sigma \to \mathbb{B})$
- **Com:** $\mathcal{C}[\![\cdot]\!] : \text{Com} \to (\Sigma \rightharpoonup \Sigma)$

Where the state Σ is a partial function from identifiers to values: $\Sigma : \text{Ide} \to \mathbb{Z}$.

2.3 Axiomatic Semantics

Based on Floyd-Hoare Logic. It uses **Hoare Triples**:

$$\{P\}\ c\ \{Q\}$$

If the precondition P is true before c, then the postcondition Q will be true after the execution of c (assuming termination).

---

3. Meta-Language Properties

3.1 Termination

A program terminates if a reachable final result exists.

$$\forall a \in \text{Aexp}, \forall \sigma \in \Sigma, \exists n \in \mathbb{Z}.\ \langle a, \sigma \rangle \to n$$

3.2 Determinism

If the computation terminates, the result is unique.

$$\forall a, \sigma, n_1, n_2.\ (\langle a, \sigma \rangle \to n_1 \wedge \langle a, \sigma \rangle \to n_2) \implies n_1 = n_2$$

3.3 Equivalence

Two programs are equivalent if they are observationally indistinguishable.

- **Operational Equivalence ($\sim$):**

$$c_0 \sim c_1 \iff \forall \sigma, \sigma'.\ (\langle c_0, \sigma \rangle \to \sigma' \iff \langle c_1, \sigma \rangle \to \sigma')$$

- **Denotational Equivalence ($\equiv_{den}$):**

$$c_0 \equiv_{den} c_1 \iff \mathcal{C}[\![c_0]\!] = \mathcal{C}[\![c_1]\!]$$

3.4 Consistency

Fundamental theorem linking OS and DS. **Theorem 6.3:** For every command c and states σ, σ':

$$\langle c, \sigma \rangle \to \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$$

The proof requires two directions:

1. **Correctness:** $\langle c, \sigma \rangle \to \sigma' \implies \mathcal{C}[\![c]\!]\sigma = \sigma'$
2. **Completeness:** $\mathcal{C}[\![c]\!]\sigma = \sigma' \implies \langle c, \sigma \rangle \to \sigma'$

---

4. Logical Tools and Induction

4.1 Logical Systems

Defined by a set of **Inference Rules**:

$$\frac{\text{Premise}_1 \quad \ldots \quad \text{Premise}_n}{\text{Conclusion}} \quad \text{(Rule Name)}$$

A **Theorem** ($\vdash_R y$) is a formula for which a finite derivation (proof tree) based on the rules exists.

4.2 Unification

The process of finding a substitution σ (MGU - Most General Unifier) such that $\sigma t_1 = \sigma t_2$. Essential for applying SOS rules and in logic programming.

4.3 Induction Principles

All principles derive from well-founded induction.

Definition (Well-Founded Relation): A relation $\prec \subseteq A \times A$ is well-founded if there are no infinite descending chains $(a_0 \succ a_1 \succ \ldots)$.

Theorem (Well-Founded Induction): To prove $\forall x \in A.\ P(x)$, it is sufficient to show:

$$\forall x \in A.\ (\forall y \prec x.\ P(y)) \implies P(x)$$

Variants of Induction

Type	Scope	Description
Mathematical	$\mathbb{N}$	Base $P(0)$, Step $P(n) \implies P(n+1)$.
Structural	Syntactic Terms	Prove P for every constructor f, assuming P for sub-terms.
Rule Induction	Derivations/Theorems	Fundamental for inductive definitions (e.g., $\rightarrow$). If the property holds for the premises of every rule, it holds for the conclusion.

> **Deep Dive:** Structural induction fails on complex recursive constructs (or when syntax is circular). In those cases, rule induction on the derivation of the semantics is strictly necessary.

---

5. Functional Preliminaries (HOFL)

5.1 λ-Notation

Used as a meta-language for functions.

- **Abstraction:** $\lambda x.e$
- **α-conversion:** Renaming of bound variables ($\lambda x.x \equiv \lambda y.y$).
- **Substitution:** $t'[t/x]$ (must be *capture-avoiding*).

5.2 Haskell (Example)

Purely functional language used to exemplify referential transparency.

```haskell
-- Example: Palindrome check
-- Demonstrates declarative style and usage of higher-order functions
pal :: Eq a => [a] -> Bool
pal xs = (xs == reverse xs)
```

Math & Logic

This chapter formalizes the necessary tools to handle recursion, prove properties of infinite systems, and constructively define semantics.

—

1. Induction Principles

The core concept is **Well-Founded Induction**, from which all other principles are derived.

1.1 Well-Founded Relations

Definition (Well-Founded Relation): A binary relation $\prec \subseteq A \times A$ is **well-founded** if it does not admit infinite descending chains $a_0 \succ a_1 \succ a_2 \ldots$.

Equivalent to: **Minimal Element Principle**. $\prec$ is well-founded iff every non-empty subset $Q \subseteq A$ has a minimal element m ($\forall x \in Q. x \not\prec m$).

1.2 Principle of Well-Founded Induction (WFI)

Let $\prec$ be a well-founded relation on A. To prove $\forall a \in A. P(a)$:

$$\forall a \in A.\ (\forall b \prec a. P(b)) \implies P(a)$$

If we prove that the property holds for a assuming it is true for all its predecessors (Inductive Hypothesis), then it holds for the entire domain.

1.3 Rule Induction

When working with logical systems defined by inference rules (such as operational semantics), structural induction is insufficient (e.g., for `while`). We use induction on derivations.

Principle: Let I_R be the set of theorems derivable in a system R. To prove that a property P holds for every theorem $y \in I_R$:

$$\frac{\forall (\frac{x_1 \ldots x_n}{y}) \in R.\ (P(x_1) \wedge \cdots \wedge P(x_n)) \implies P(y)}{\forall \delta \in I_R. P(\delta)}$$

If the property is preserved by every rule (from premises to conclusion), then it holds for all derived theorems.

Practical Example: Determinacy of IMP We want to prove that if a command terminates, the result is unique. **Theorem:** $\langle c, \sigma \rangle \to \sigma_1 \wedge \langle c, \sigma \rangle \to \sigma_2 \implies \sigma_1 = \sigma_2$.

We define the property P on the derivation $\langle c, \sigma \rangle \to \sigma_1$ as:

$$P(\langle c, \sigma \rangle \to \sigma_1) \iff \forall \sigma_2.\ \langle c, \sigma \rangle \to \sigma_2 \implies \sigma_1 = \sigma_2$$

We apply rule induction:

1. **Skip Rule:** $\langle \textbf{skip}, \sigma \rangle \to \sigma$. If we have another derivation $\langle \textbf{skip}, \sigma \rangle \to \sigma_2$, the same rule must have been used (it is the only one for skip). Thus $\sigma_2 = \sigma$. (OK)
2. **Seq Rule:** $\frac{\langle c_0, \sigma \rangle \to \sigma'' \quad \langle c_1, \sigma'' \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \sigma'}$. *Inductive Hypothesis (IH):* Determinacy holds for the premises (execution of c_0 and c_1). *Step:* If $\langle c_0; c_1, \sigma \rangle \to \sigma_2$, the only applicable rule is Seq. Thus there exists σ''_2 such that $\langle c_0, \sigma \rangle \to \sigma''_2$ and $\langle c_1, \sigma''_2 \rangle \to \sigma_2$. By IH on c_0: $\sigma'' = \sigma''_2$. By IH on c_1: since the intermediate state is the same, $\sigma' = \sigma_2$. (OK)

---

2. Immediate Consequence Operator (ICO)

The set of theorems I_R of a logical system can be viewed as the fixpoint of a function. This connects logic to domain theory.

2.1 Definition of $\hat{R}$

Let F be the set of all possible formulas (facts). The operator $\hat{R} : \wp(F) \to \wp(F)$ computes the facts derivable in **one single step** from a set of hypotheses S.

$$\hat{R}(S) \triangleq \left\{ y \mid \exists \frac{x_1 \dots x_n}{y} \in R \text{ such that } \{x_1, \dots, x_n\} \subseteq S \right\}$$

2.2 Characterization Theorem

The operator $\hat{R}$ is **monotone** and **continuous** (if rules have finite premises). The set of theorems I_R coincides with the least fixpoint of $\hat{R}$:

$$I_R = \text{fix}(\hat{R}) = \bigcup_{n \in \mathbb{N}} \hat{R}^n(\emptyset)$$

- $\hat{R}^0(\emptyset) = \emptyset$
- $\hat{R}^1(\emptyset) =$ Axioms (rules without premises).
- $\hat{R}^{n+1}(\emptyset) =$ Facts derivable with proof trees of height $\leq n+1$.

2.3 Example: Balanced Parentheses

Consider the grammar for strings of balanced parentheses: $\epsilon \mid (s) \mid s_1 s_2$. Rules R:

1. $\overline{\epsilon}$ (Axiom)
2. $\frac{s}{(s)}$
3. $\frac{s_1 \quad s_2}{s_1 s_2}$

We compute the fixpoint (set of theorems) by iterating $\hat{R}$:

1. **Step 0:** $S_0 = \emptyset$.
2. **Step 1:** $\hat{R}(S_0) = \{\epsilon\}$ (Applying rule 1).
3. **Step 2:** $\hat{R}(S_1)$ uses $S_1 = \{\epsilon\}$.
   - Rule 2 on $\epsilon \to (\epsilon) = ()$.
   - Rule 3 on $\epsilon, \epsilon \to \epsilon\epsilon = \epsilon$.
   - $S_2 = \{\epsilon, ()\}$.
4. **Step 3:** $\hat{R}(S_2)$ uses $\{\epsilon, ()\}$.
   - Rule 2 on $() \to (())$.
   - Rule 3 on $s_1 = (), s_2 = () \to ()()$.
   - $S_3 = \{\epsilon, (), (()), ()()\}$.
5. **Limit:** The union $\bigcup_n S_n$ is the set of all balanced strings.

---

3. Partial Orders and CPO

3.1 Basic Definitions

- **Partial Order (PO):** Set P with a relation $\sqsubseteq$ that is reflexive, antisymmetric, and transitive.
- **Chain:** A sequence $\{d_n\}_{n \in \mathbb{N}}$ that is totally ordered: $d_0 \sqsubseteq d_1 \sqsubseteq d_2 \dots$
- **Limit (LUB):** The *Least Upper Bound* of a chain, denoted as $\bigsqcup_n d_n$.

3.2 Complete Partial Orders (CPO)

Definition (CPO): A PO is a **CPO** if every chain has a limit (LUB) in D.

Definition (CPO$\perp$): A CPO is **pointed** (CPO$\perp$) if it possesses a minimal element, called **bottom** ($\perp$).

3.3 Functions on CPO

Let D, E be two CPOs. A function $f : D \to E$ can be:

1. **Monotone:** Preserves the order ($d \sqsubseteq d' \implies f(d) \sqsubseteq f(d')$).
2. **Continuous:** Preserves the limits of chains.

$$f\left(\bigsqcup_n d_n\right) = \bigsqcup_n f(d_n)$$

Note: Continuity implies monotonicity.

---

4. Kleene's Fixpoint Theorem

4.1 Statement

Let D be a CPO$\perp$ and $f : D \to D$ a **continuous** function. Then f has a **least fixed point**, computable as:

$$\text{fix}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\perp)$$

Where the approximation chain is: $\perp \sqsubseteq f(\perp) \sqsubseteq f(f(\perp)) \ldots$

4.2 Proof (Sketch)

1. **Chain:** Prove by mathematical induction that $f^n(\perp) \sqsubseteq f^{n+1}(\perp)$. (Base: $\perp \sqsubseteq f(\perp)$ is obvious).
2. **Fixpoint:** Let $d = \bigsqcup f^n(\perp)$. Applying f:

$$f(d) = f(\bigsqcup f^n(\perp)) = \bigsqcup f(f^n(\perp)) = \bigsqcup f^{n+1}(\perp) = d$$

   Continuity allows "moving the limit inside".
3. **Least:** If e is another fixpoint ($f(e) = e$), prove by induction that $f^n(\perp) \sqsubseteq e$, thus the limit $d \sqsubseteq e$.

---

5. Solved Exercises (Domain Theory)

Exercise: Continuity on Power Sets

Problem: Consider the CPO $(\wp(\mathbb{N}), \subseteq)$. Let $S \subseteq \mathbb{N}$ be a fixed set. Prove that the function $f_S : \wp(\mathbb{N}) \to \wp(\mathbb{N})$ defined as $f_S(X) = X \cap S$ is **continuous**.

Solution: To prove continuity, we must show that f_S preserves the limits of chains (Least Upper Bounds). In $(\wp(\mathbb{N}), \subseteq)$, the limit of a chain is the union of the sets: $\bigsqcup_i X_i = \bigcup_{i \in \mathbb{N}} X_i$.

We need to demonstrate that for every chain $\{X_i\}_{i \in \mathbb{N}}$:

$$f_S\left(\bigcup_{i \in \mathbb{N}} X_i\right) = \bigcup_{i \in \mathbb{N}} f_S(X_i)$$

Proof:

1. **LHS (Left Hand Side):**

$$f_S\left(\bigcup_i X_i\right) = \left(\bigcup_i X_i\right) \cap S$$

2. **RHS (Right Hand Side):**
$$\bigcup_i f_S(X_i) = \bigcup_i (X_i \cap S)$$
3. **Conclusion:** By the distributive property of intersection over union (including infinite union), we know that:
$$(\bigcup_i X_i) \cap S = \bigcup_i (X_i \cap S)$$
Thus LHS = RHS. The function is continuous.

   Note: The same logic applies to $g_S(X) = X \cup S$ (using associativity/idempotence of union).

IMP Semantics

This chapter defines the semantics of the **IMP** language, a minimal imperative language with static memory.

---

1. Syntax and States

Abstract Syntax

The language is defined by three syntactic categories (*sorts*):

- **Aexp:** Arithmetic Expressions (values in $\mathbb{Z}$)
- **Bexp:** Boolean Expressions (values in $\mathbb{B}$)
- **Com:** Commands (state transformations)

$$\begin{array}{lcl} a \in \text{Aexp} & ::= & n \mid x \mid a_0 + a_1 \mid a_0 - a_1 \mid \ldots \\ b \in \text{Bexp} & ::= & v \mid a_0 = a_1 \mid a_0 \leq a_1 \mid \neg b \mid b_0 \wedge b_1 \ldots \\ c \in \text{Com} & ::= & \textbf{skip} \mid x := a \mid c_0; c_1 \mid \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1 \mid \textbf{while } b \textbf{ do } c \end{array}$$

The State (Σ)

The state (or memory) is a function mapping identifiers to integer values.

$$\Sigma \stackrel{\text{def}}{=} \text{Loc} \to \mathbb{Z}$$

- **Update Notation:** $\sigma[n/x]$ denotes a state identical to σ except for the variable x, which now holds the value n.

---

2. Operational Semantics (Big-Step)

Defined via inference rules relating a program and an initial state to a final result (hiding intermediate steps). This style is also known as **Natural Semantics**.

Judgments

1. $\langle a, \sigma \rangle \to n$ (Aexp always terminates)
2. $\langle b, \sigma \rangle \to v$ (Bexp always terminates)
3. $\langle c, \sigma \rangle \to \sigma'$ (Convergence: the command terminates producing σ')

Inference Rules for Commands

Skip and Assignment:

$$\frac{}{\langle \textbf{skip}, \sigma\rangle \to \sigma}(\text{skip}) \qquad \frac{\langle a, \sigma\rangle \to m}{\langle x := a, \sigma\rangle \to \sigma[m/x]}(\text{assign})$$

Sequence:

$$\frac{\langle c_0, \sigma\rangle \to \sigma'' \quad \langle c_1, \sigma''\rangle \to \sigma'}{\langle c_0; c_1, \sigma\rangle \to \sigma'}(\text{seq})$$

Conditional (If-Then-Else):

$$\frac{\langle b, \sigma\rangle \to \textbf{true} \quad \langle c_0, \sigma\rangle \to \sigma'}{\langle \textbf{if } b \ldots, \sigma\rangle \to \sigma'}(\text{iftt}) \quad \frac{\langle b, \sigma\rangle \to \textbf{false} \quad \langle c_1, \sigma\rangle \to \sigma'}{\langle \textbf{if } b \ldots, \sigma\rangle \to \sigma'}(\text{ifff})$$

While (Iteration):

$$\frac{\langle b, \sigma\rangle \to \textbf{false}}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \to \sigma}(\text{whff})$$

$$\frac{\langle b, \sigma\rangle \to \textbf{true} \quad \langle c, \sigma\rangle \to \sigma'' \quad \langle \textbf{while } b \textbf{ do } c, \sigma''\rangle \to \sigma'}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \to \sigma'}(\text{whtt})$$

> **Note:** The `whtt` rule is the only intrinsically recursive rule (the premise contains the same construct as the conclusion). This makes structural induction insufficient for proving properties about `while`; **rule induction** is required.

---

3. Denotational Semantics

Assigns meanings as mathematical functions. Since commands may diverge (infinite loop), we use domains with an undefined element (Bottom).

Domains and Functions

- **Lifted State ($\Sigma_\perp$):** $\Sigma \cup \{\perp\}$. The element $\perp$ represents non-termination.
- $\mathcal{A}[\![a]\!] : \Sigma \to \mathbb{Z}$
- $\mathcal{B}[\![b]\!] : \Sigma \to \mathbb{B}$
- $\mathcal{C}[\![c]\!] : \Sigma \to \Sigma_\perp$ (Total function returning $\perp$ if it diverges).

Lifting (Strict Extension)

To compose functions that may return $\perp$, we define the lifting operator $(\cdot)^*$: Given $f : \Sigma \to \Sigma_\perp$, its extension $f^* : \Sigma_\perp \to \Sigma_\perp$ is:

$$f^*(x) = \begin{cases} \perp & \text{if } x = \perp \\ f(x) & \text{if } x \in \Sigma \end{cases}$$

This is crucial for sequence: $\mathcal{C}[\![c_0; c_1]\!]\sigma = \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma)$.

Denotation of While (Fixpoint)

The `while` construct is defined as the **Least Fixed Point** (LFP) of a functional Γ.

$$\mathcal{C}[\![\textbf{while } b \textbf{ do } c]\!] = \text{fix}(\Gamma_{b,c})$$

Where $\Gamma_{b,c} : (\Sigma \to \Sigma_\perp) \to (\Sigma \to \Sigma_\perp)$ is defined as:

$$\Gamma_{b,c}(\varphi) = \lambda\sigma.\text{cond}(\mathcal{B}[\![b]\!]\sigma, \varphi^*(\mathcal{C}[\![c]\!]\sigma), \sigma)$$

Applying Kleene's Theorem:

$$\text{fix}(\Gamma) = \bigsqcup_{n\in\mathbb{N}} \Gamma^n(\perp)$$

Where $\Gamma^n(\perp)$ represents the semantics of the loop limited to n iterations.

---

4. Equivalence and Consistency

Operational Equivalence

Two commands are equivalent ($c_1 \sim c_2$) if they are observationally identical:

$$c_1 \sim c_2 \iff \forall\sigma, \sigma'.(\langle c_1, \sigma\rangle \to \sigma' \iff \langle c_2, \sigma\rangle \to \sigma')$$

Consistency Theorem (Theorem 6.3)

Operational Semantics (Big-Step) and Denotational Semantics coincide for every terminating program.

Statement:

$$\forall c \in \text{Com}, \forall\sigma, \sigma' \in \Sigma. \quad (\langle c, \sigma\rangle \to \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma')$$

The proof is divided into two parts (Correctness and Completeness).

Part 1: Correctness ($\Rightarrow$) **Goal:** Prove that $\langle c, \sigma\rangle \to \sigma' \implies \mathcal{C}[\![c]\!]\sigma = \sigma'$. **Technique:** Induction on the inference rules of the operational semantics (Rule Induction).

We define the property $P(\text{rule}) \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$.

1. **Base Case (Skip):** Rule is $\langle \textbf{skip}, \sigma\rangle \to \sigma$. By denotational definition: $\mathcal{C}[\![\textbf{skip}]\!]\sigma = \sigma$. Thesis verified.
2. **Base Case (Assign):** Rule concludes $\langle x := a, \sigma\rangle \to \sigma[m/x]$ with premise $\langle a, \sigma\rangle \to m$. By consistency of expressions, $\mathcal{A}[\![a]\!]\sigma = m$. The denotational semantics defines $\mathcal{C}[\![x := a]\!]\sigma = \sigma[\mathcal{A}[\![a]\!]\sigma/x]$, which coincides with $\sigma[m/x]$.
3. **Inductive Step (Seq):** Rule: $\frac{\langle c_0,\sigma\rangle\to\sigma'' \quad \langle c_1,\sigma''\rangle\to\sigma'}{\langle c_0;c_1,\sigma\rangle\to\sigma'}$. Inductive Hypotheses: $\mathcal{C}[\![c_0]\!]\sigma = \sigma''$ and $\mathcal{C}[\![c_1]\!]\sigma'' = \sigma'$. Thesis:
$$\begin{aligned}\mathcal{C}[\![c_0; c_1]\!]\sigma &= \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma)\\ &= \mathcal{C}[\![c_1]\!]^*(\sigma'') \quad \text{(by IH on } c_0)\\ &= \mathcal{C}[\![c_1]\!]\sigma'' \quad \text{(since } \sigma'' \neq \perp)\\ &= \sigma' \quad \text{(by IH on } c_1)\end{aligned}$$
.
4. **Inductive Step (While - True Case):** Rule `whtt`. Assume $\mathcal{B}[\![b]\!]\sigma = \textbf{true}$. Inductive Hypotheses: $\mathcal{C}[\![c]\!]\sigma = \sigma''$ and $\mathcal{C}[\![\textbf{while } \ldots]\!]\sigma'' = \sigma'$. Thesis: Using the fixpoint property ($\text{fix}(\Gamma) = \Gamma(\text{fix}(\Gamma))$):
$$\begin{aligned}\mathcal{C}[\![w]\!]\sigma &= \mathcal{B}[\![b]\!]\sigma \to \mathcal{C}[\![w]\!]^*(\mathcal{C}[\![c]\!]\sigma), \sigma\\ &= \textbf{true} \to \mathcal{C}[\![w]\!]^*(\sigma''), \sigma\\ &= \mathcal{C}[\![w]\!]\sigma'' = \sigma' \quad \text{(by IH)}\end{aligned}$$

Part 2: Completeness ($\Leftarrow$) Goal: Prove that $\mathcal{C}[\![c]\!]\sigma = \sigma' \implies \langle c, \sigma\rangle \to \sigma'$ (assuming $\sigma' \neq \bot$). **Technique:** Structural Induction on c.

1. **Simple Cases:** For `skip`, `assign`, `seq`, and `if`, the proof is specular to correctness.
2. **While Case (`w`):** Structural induction fails because $\mathcal{C}[\![w]\!]$ is defined as a fixpoint. We use **Mathematical Induction** on the index n of the fixpoint approximations:

$$\mathcal{C}[\![w]\!]\sigma = \bigsqcup_n \Gamma^n(\bot)\sigma$$

We define property $A(n)$: $\forall \sigma, \sigma'. \Gamma^n(\bot)\sigma = \sigma' \implies \langle w, \sigma\rangle \to \sigma'$.

- **Base ($n = 0$):** $\Gamma^0(\bot)\sigma = \bot$. Since the premise requires $\sigma' \neq \bot$, the implication is vacuously true.
- **Step ($n + 1$):** Assume $A(n)$ holds. Let $\Gamma^{n+1}(\bot)\sigma = \sigma'$. Expanding Γ:

$$\Gamma^{n+1}(\bot)\sigma = \text{cond}(\mathcal{B}[\![b]\!]\sigma, \Gamma^n(\bot)^*(\mathcal{C}[\![c]\!]\sigma), \sigma)$$

  - If b is **false**: The result is σ. Rule `whff` confirms $\langle w, \sigma\rangle \to \sigma$.
  - If b is **true**: Then $\Gamma^n(\bot)(\sigma'') = \sigma'$, where $\sigma'' = \mathcal{C}[\![c]\!]\sigma$. By structural induction hypothesis on c: $\langle c, \sigma\rangle \to \sigma''$. By mathematical induction hypothesis on n: $\langle w, \sigma''\rangle \to \sigma'$. Applying rule `whtt`: $\langle w, \sigma\rangle \to \sigma'$.

---

5. Derivation Example (Swap)

Big-Step derivation for the variable swap program.

Initial State: $\sigma = \{x \mapsto 10, y \mapsto 20, z \mapsto 0\}$ **Program:** $c \equiv z := x; (x := y; y := z)$

Define intermediate states:

1. $\sigma_1 = \sigma[10/z] = \{x \mapsto 10, y \mapsto 20, z \mapsto 10\}$
2. $\sigma_2 = \sigma_1[20/x] = \{x \mapsto 20, y \mapsto 20, z \mapsto 10\}$
3. $\sigma_{final} = \sigma_2[10/y] = \{x \mapsto 20, y \mapsto 10, z \mapsto 10\}$

Derivation Tree:

$$\dfrac{\dfrac{\langle x, \sigma\rangle \to 10}{\langle z := x, \sigma\rangle \to \sigma_1}(\text{ass}) \quad \dfrac{\dfrac{\langle y, \sigma_1\rangle \to 20}{\langle x := y, \sigma_1\rangle \to \sigma_2}(\text{ass}) \quad \dfrac{\langle z, \sigma_2\rangle \to 10}{\langle y := z, \sigma_2\rangle \to \sigma_{final}}(\text{ass})}{\langle x := y; y := z, \sigma_1\rangle \to \sigma_{final}}(\text{seq})}{\langle z := x; (x := y; y := z), \sigma\rangle \to \sigma_{final}}(\text{seq})$$

—

6. Solved Exercises (Exam)

Exercise: Efficiency Measure (Counting Guards)

Source: Exam 19/06/2019

Problem: We want to insert an efficiency measure into the operational semantics of IMP.

1. Redefine the semantics such that the judgment $\langle c, \sigma\rangle \to \sigma'$ becomes $\langle c, \sigma\rangle \xrightarrow{k} \sigma'$, where k is the exact number of boolean guards evaluated during execution.
2. Prove by induction that if the standard semantics converges, then there exists a k for which the new semantics converges.

Solution

1. New Operational Rules ($\xrightarrow{k}$) The idea is that every time we evaluate a `Bexp` (in `if` or `while`), the cost k increases by 1. Structural commands sum the costs of sub-commands.

- **Skip / Assign:** No guard evaluated.

$$\frac{}{\langle \textbf{skip}, \sigma\rangle \xrightarrow{0} \sigma} \qquad \frac{\langle a, \sigma \to n\rangle}{\langle x := a, \sigma\rangle \xrightarrow{0} \sigma[n/x]}$$

- **Sequence:** Sum of costs.

$$\frac{\langle c_0, \sigma\rangle \xrightarrow{k_1} \sigma'' \quad \langle c_1, \sigma''\rangle \xrightarrow{k_2} \sigma'}{\langle c_0; c_1, \sigma\rangle \xrightarrow{k_1+k_2} \sigma'}$$

- **If-Then-Else:** Cost 1 (for the guard) + cost of the chosen branch.

$$\frac{\langle b, \sigma\rangle \to \textbf{true} \quad \langle c_0, \sigma\rangle \xrightarrow{k} \sigma'}{\langle \textbf{if } b \ldots, \sigma\rangle \xrightarrow{k+1} \sigma'} \qquad \frac{\langle b, \sigma\rangle \to \textbf{false} \quad \langle c_1, \sigma\rangle \xrightarrow{k} \sigma'}{\langle \textbf{if } b \ldots, \sigma\rangle \xrightarrow{k+1} \sigma'}$$

- **While:**
  - *False Case:* Evaluate guard (1) and exit.

$$\frac{\langle b, \sigma\rangle \to \textbf{false}}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \xrightarrow{1} \sigma}$$

  - *True Case:* Evaluate guard (1) + body cost (k_c) + subsequent iterations cost (k_w).

$$\frac{\langle b, \sigma\rangle \to \textbf{true} \quad \langle c, \sigma\rangle \xrightarrow{k_c} \sigma'' \quad \langle \textbf{while } \ldots, \sigma''\rangle \xrightarrow{k_w} \sigma'}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \xrightarrow{1+k_c+k_w} \sigma'}$$

2. Proof (Rule Induction) We want to prove: $\langle c, \sigma\rangle \to \sigma' \implies \exists k \in \mathbb{N}.\ \langle c, \sigma\rangle \xrightarrow{k} \sigma'$. We define the property $P(\langle c, \sigma\rangle \to \sigma') \iff \exists k.\ \langle c, \sigma\rangle \xrightarrow{k} \sigma'$.

We apply **Rule Induction** on the standard semantics:

- **Seq Case:**
  - *Rule:* $\frac{\langle c_0,\sigma\rangle\to\sigma'' \quad \langle c_1,\sigma''\rangle\to\sigma'}{\ldots}$
  - *Inductive Hypotheses:* There exist k_1, k_2 such that $\langle c_0, \sigma\rangle \xrightarrow{k_1} \sigma''$ and $\langle c_1, \sigma''\rangle \xrightarrow{k_2} \sigma'$.
  - *Thesis:* We can apply the new semantics rule with cost $k = k_1 + k_2$. Thus a k exists. (Verified).
- **While-True Case:**
  - *Rule:* Premises $\langle b, \sigma\rangle \to \textbf{true}$, $\langle c, \sigma\rangle \to \sigma''$, $\langle w, \sigma''\rangle \to \sigma'$.
  - *Inductive Hypotheses:* 1. $\exists k_c$ s.t. $\langle c, \sigma\rangle \xrightarrow{k_c} \sigma''$ 2. $\exists k_w$ s.t. $\langle w, \sigma''\rangle \xrightarrow{k_w} \sigma'$
  - *Thesis:* Using the new while-true rule, we can derive the transition with cost $k = 1 + k_c + k_w$. Since $k_c, k_w \in \mathbb{N}$, then $k \in \mathbb{N}$. (Verified).

The other cases (`skip`, `assign`, `if`, `while-false`) are trivial or analogous.

HOFL

HOFL (*Higher-Order Functional Language*) introduces functions as "first-class citizens". The complexity shifts from state management (which is absent) to the handling of complex types, recursion, and infinite domains.

---

1. Syntax and Types

1.1 Term Syntax

The grammar includes constructs for arithmetic, conditionals, pairs, functions, and recursion.

$$t ::= x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \mid \lambda x.t \mid t_0 t_1 \mid \text{rec } x.t$$

1.2 Type System

Types are defined inductively:

$$\tau ::= \text{int} \mid \tau_0 \times \tau_1 \mid \tau_0 \to \tau_1$$

A term is **well-typed** $(t : \tau)$ if a derivation exists using the following inference rules $(\Gamma \vdash t : \tau)$:

- **Abstraction (abs):**
$$\frac{\Gamma, x : \tau_0 \vdash t : \tau_1}{\Gamma \vdash \lambda x.t : \tau_0 \to \tau_1}$$
- **Application (app):**
$$\frac{\Gamma \vdash t_1 : \tau_0 \to \tau_1 \quad \Gamma \vdash t_0 : \tau_0}{\Gamma \vdash t_1 t_0 : \tau_1}$$
- **Recursion (rec):**
$$\frac{\Gamma, x : \tau \vdash t : \tau}{\Gamma \vdash \text{rec } x.t : \tau}$$

---

2. Operational Semantics (Lazy)

In HOFL, there is no state (Environment Model). Evaluation is a relation $t \to c$ where c is a **Canonical Form**.

2.1 Canonical Forms (C_τ)

Values that require no further computation (final results).

1. **Integers:** $n \in \mathbb{Z}$.
2. **Pairs:** (t_0, t_1) where t_0, t_1 are closed terms (not necessarily evaluated in Lazy semantics).
3. **Abstractions:** $\lambda x.t$ (functions are values; the body is not evaluated until applied).

2.2 Inference Rules (Big-Step Lazy)

Arithmetic and Conditional:

$$\frac{t_0 \to n_0 \quad t_1 \to n_1}{t_0 \text{ op } t_1 \to n_0 \underline{\text{op}} n_1} \qquad \frac{t \to 0 \quad t_0 \to c_0}{\text{if } t \cdots \to c_0} \qquad \frac{t \to n \neq 0 \quad t_1 \to c_1}{\text{if } t \cdots \to c_1}$$

Pairs and Projections:

$$\frac{}{(t_0, t_1) \to (t_0, t_1)} \qquad \frac{t \to (t_0, t_1) \quad t_0 \to c_0}{\text{fst}(t) \to c_0}$$

Application (Lazy / Call-by-Name):

$$\frac{t_1 \to \lambda x.t_1' \quad t_1'[t_0/x] \to c}{(t_1 t_0) \to c}$$

Note: The argument t_0 is substituted into the function body **without being evaluated**.

---

3. Domain Theory

To define denotational semantics (especially for `rec`), domains must be **CPOs** (Complete Partial Orders) with a minimal element $\perp$ (bottom) representing non-termination.

3.1 Lifted Domains ($D_\perp$)

To distinguish between a computed result and non-termination, we use *lifted* domains.

$$D_\perp \triangleq \{\perp\} \cup \{\lfloor d \rfloor \mid d \in D\}$$

- $\perp$: No information (divergence).
- $\lfloor d \rfloor$: Defined value d (convergence).

3.2 Functions on Lifted Domains

1. **Lifting Function:** $\lfloor \cdot \rfloor : D \to D_\perp$. Maps $d \mapsto \lfloor d \rfloor$.
2. **Lifting Operator $(\cdot)^*$:** Extends a continuous function $f : D \to E$ to a function $f^* : D_\perp \to E$.

$$f^*(x) = \begin{cases} \perp_E & \text{if } x = \perp_{D_\perp} \\ f(d) & \text{if } x = \lfloor d \rfloor \end{cases}$$

3. **De-lifting (Let notation):** Syntactic sugar for handling lifted values.

$$(\text{let } x \leftarrow t.e) \equiv (\lambda x.e)^*(t)$$

If t diverges ($\perp$), the whole expression diverges. If $t = \lfloor d \rfloor$, it evaluates e with $x = d$.

---

4. Denotational Semantics

We define semantic domains D_τ for each type τ. In HOFL (Lazy), all domains are **Lifted** to allow divergence at every level.

4.1 Domain Definition

1. **Integers:** $D_{\text{int}} = \mathbb{Z}_\perp$ (Flat CPO).
2. **Pairs:** $D_{\tau_1 \times \tau_2} = (D_{\tau_1} \times D_{\tau_2})_\perp$.
   - *Why lifted?* To distinguish a diverging pair $\perp$ from a pair of diverging elements $(\perp, \perp)$.
3. **Functions:** $D_{\tau_1 \to \tau_2} = [D_{\tau_1} \to D_{\tau_2}]_\perp$.
   - Space of **continuous** functions between CPOs, lifted to distinguish the undefined function $\perp$ from the function that always diverges $\lambda x.\ \perp$.

4.2 Interpretation Function $[\![t]\!]\rho$

Maps terms and environments to elements of the domain: $[\![t]\!] : \text{Env} \to D_\tau$.

Basic Constructs:

- $[\![n]\!]\rho = \lfloor n \rfloor$
- $[\![x]\!]\rho = \rho(x)$
- $[\![t_1 \text{ op } t_2]\!]\rho = [\![t_1]\!]\rho \underline{\text{op}}_\perp [\![t_2]\!]\rho$ (Strict extension: if one is $\perp$, result is $\perp$).

Functional Constructs:

- **Abstraction (λ):**

$$[\![\lambda x.t]\!]\rho = \lfloor \lambda d.[\![t]\!]\rho[d/x] \rfloor$$

The result is a defined element ($\lfloor \ldots \rfloor$) in the functional domain.

- **Application:**

$$[\![t_1 t_0]\!]\rho = \text{let } \varphi \leftarrow [\![t_1]\!]\rho.\varphi([\![t_0]\!]\rho)$$

  1. Evaluate t_1. If it diverges, return $\perp$.
  2. If it converges to a function φ, apply φ to the denotation of the argument $[\![t_0]\!]\rho$ (without evaluating it first $\rightarrow$ Lazy).

Recursion (`rec`):

$$[\![\text{rec } x.t]\!]\rho = \text{fix}(\lambda d.[\![t]\!]\rho[d/x])$$

Computes the least fixed point of the function mapping x to the body t.

---

5. Consistency and Comparison

5.1 Correctness Theorem

Denotational semantics is consistent with operational semantics.

$$t \rightarrow c \implies \forall \rho.\ [\![t]\!]\rho = [\![c]\!]\rho$$

5.2 Lazy vs Eager (Unlifted)

In an **Eager** (Call-by-Value) language, the argument must be evaluated before application. This changes the domain structure.

Feature	Lazy (Standard HOFL)	Eager (CbyV)
App Rule	Substitutes un-evaluated t_0.	Evaluates $t_0 \rightarrow c_0$, then substitutes c_0.
Pair Domain	$(D_1 \times D_2)_\perp$ (Lifted)	$D_1 \times D_2$ (Unlifted - "smashed" product)
Function Domain	$[D_1 \rightarrow D_2]_\perp$	$[D_1 \rightarrow D_2]$
Divergence	`(\x. 1) (rec y. y)` $\rightarrow 1$	`(\x. 1) (rec y. y)` diverges.

> **Note on Unlifted Domains:** In Eager semantics, lifting is not needed on compound types because a term of type τ *always* denotes a defined value or diverges *before* returning. Domains become: $U_{\text{int}} = \mathbb{Z}_\perp$, $U_{\tau_1 \times \tau_2} = U_{\tau_1} \times U_{\tau_2}$.

Concurrency

This chapter moves from sequential models to concurrent ones. The focus is no longer on "computing a function", but on **interaction**, **non-determinism**, and **communication**.

---

1. Language Overview: Erlang vs Go

Both use *Message Passing*, but with opposite philosophies regarding memory and synchronization.

Feature	**Erlang**	**Go**
Concurrent Entity	Process (Actor) isolated (separate heap).	Goroutine (Lightweight Thread) with shared memory.
Communication	**Asynchronous** (Infinite Mailbox). Sender does not block.	**Synchronous** (default). `ch <- v` blocks until there is a receiver (Rendezvous).
Philosophy	"Let it crash" & Isolation.	"Do not communicate by sharing memory; share memory by communicating."
Reception	`receive` with Pattern Matching on the mailbox.	`<- ch` (select for multiple wait).

---

2. CCS (Calculus of Communicating Systems)

Process algebra introduced by Robin Milner. It is Turing-equivalent but focused on pure interaction (abstracts away from data values).

2.1 Process Syntax

$$P, Q ::= \mathbf{nil} \mid \alpha.P \mid P + Q \mid P \mid Q \mid P \setminus L \mid P[f] \mid A$$

- **Prefix ($\alpha.P$):** Performs action α and becomes P. α can be an input (a), an output ($\bar{a}$), or an internal action (τ).
- **Sum ($P + Q$):** Non-deterministic choice. If P acts, Q is discarded (and vice versa).
- **Parallel ($P \mid Q$):** P and Q execute concurrently (interleaving or synchronization).
- **Restriction ($P \setminus L$):** Actions in L (and their duals $\bar{L}$) become private: they cannot be performed externally but allow internal communication.
- **Identifier (A):** Used to define recursive processes (e.g., $A \stackrel{\text{def}}{=} \alpha.A$).

2.2 Operational Semantics (LTS)

Transitions are of the form $P \xrightarrow{\mu} P'$. The set of actions is $\text{Act} = \mathcal{L} \cup \{\tau\}$, where τ is the invisible (internal) action.

Dynamic Rules:

$$\frac{}{\alpha.P \xrightarrow{\alpha} P}(\text{Act}) \quad \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'}(\text{SumL}) \quad \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}(\text{SumR})$$

Static Rules (Parallel and Communication): Parallel composition allows interleaving (independent actions) or synchronization (handshake).

$$\frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}(\text{ParL}) \quad \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}(\text{ParR})$$

Communication Rule (Synchronization): For communication to occur, P must perform an output (a) and Q an input ($\bar{a}$). The result is an internal action τ.

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}(\text{Com})$$

Restriction Rule: Blocks actions in L, allowing only unrestricted actions or τ (result of internal communication) to pass.

$$\frac{P \xrightarrow{\mu} P' \quad \mu \notin L \cup \bar{L}}{P \setminus L \xrightarrow{\mu} P' \setminus L}(\text{Res})$$

2.3 Example of LTS Derivation

Consider the process $S \stackrel{\text{def}}{=} (a.P \mid \bar{a}.Q) \setminus \{a\}$. We want to derive the transition $S \xrightarrow{\tau} (P \mid Q) \setminus \{a\}$.

Derivation Tree:

$$\frac{\dfrac{\dfrac{}{\alpha.P \xrightarrow{a} P}(\text{Act}) \quad \dfrac{}{\bar{a}.Q \xrightarrow{\bar{a}} Q}(\text{Act})}{a.P|\bar{a}.Q \xrightarrow{\tau} P|Q}(\text{Com})}{(a.P|\bar{a}.Q)\setminus\{a\} \xrightarrow{\tau} (P|Q)\setminus\{a\}}(\text{Res})$$

Note: Without restriction, the process could also perform a (left only) or $\bar{a}$ (right only). Restriction "forces" synchronization by making individual communication attempts on a invisible.

---

3. Bisimulation

Trace equivalence (sequences of actions) is insufficient for concurrency because it ignores choice points (*branching structure*). Classic example: $a.(b + c)$ vs $a.b + a.c$ have the same traces $\{ab, ac\}$ but different behavior (moment of choice).

3.1 Formal Definition (Strong Bisimulation)

A binary relation $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ is a **strong bisimulation** if, for every pair $(P, Q) \in \mathcal{R}$ and for every action α:

1. **Simulation P $\rightarrow$ Q:** If $P \xrightarrow{\alpha} P'$, then $\exists Q'$ such that $Q \xrightarrow{\alpha} Q'$ and $(P', Q') \in \mathcal{R}$.
2. **Simulation Q $\rightarrow$ P:** If $Q \xrightarrow{\alpha} Q'$, then $\exists P'$ such that $P \xrightarrow{\alpha} P'$ and $(P', Q') \in \mathcal{R}$.

Two processes are **Bisimilar** ($P \sim Q$) if there exists a bisimulation $\mathcal{R}$ containing the pair (P, Q).

3.2 The Game (Attacker vs Defender)

Bisimulation can be viewed as a turn-based game between **Alice (Attacker)** and **Bob (Defender)**.

- **State:** A pair of processes (P, Q).
- **Alice:** Wants to prove $P \not\sim Q$. Chooses one side (e.g., P) and makes a transition ($P \xrightarrow{\alpha} P'$).
- **Bob:** Must prove they are equivalent. Must respond on the other side (Q) with the *same* action ($Q \xrightarrow{\alpha} Q'$) trying to land in a state that remains equivalent.
- **Victory:** Alice wins if Bob cannot respond or if the new pair of states allows Alice to win in the future. Bob wins if the game continues infinitely (bisimulation).

3.3 Example: $a.(b + c)$ vs $a.b + a.c$

We prove that $P = a.(b.\mathbf{nil} + c.\mathbf{nil})$ and $Q = a.b.\mathbf{nil} + a.c.\mathbf{nil}$ are **NOT** bisimilar.

Alice's Winning Strategy:

1. **Alice** chooses Q (the non-deterministic process) and makes the left branch transition:

$$Q \xrightarrow{a} b.\mathbf{nil}$$

We are now in the pair $(b.\mathbf{nil} + c.\mathbf{nil}, \quad b.\mathbf{nil})$.

2. **Bob** must respond with $P \xrightarrow{a}$. P has only one choice:

$$P \xrightarrow{a} b.\mathbf{nil} + c.\mathbf{nil}$$

   The new configuration is: $(b.\mathbf{nil} + c.\mathbf{nil}, \quad b.\mathbf{nil})$.
3. **Alice** now chooses the left process and performs action c:

$$(b.\mathbf{nil} + c.\mathbf{nil}) \xrightarrow{c} \mathbf{nil}$$

4. **Bob** must respond with the right process ($b.\mathbf{nil}$) performing c. But $b.\mathbf{nil}$ can only perform b. **Bob is stuck. Alice wins.** $\implies P \not\sim Q$.

3.4 Congruence

Strong bisimulation is a **congruence**: if $P \sim Q$, then $C[P] \sim C[Q]$ for any context C. This allows modular substitution of software components without altering system behavior.

---

4. Hennessy-Milner Logic (HML)

Modal logic for describing process properties. It is used as an alternative to bisimulation to distinguish processes.

4.1 Syntax

$$F ::= \text{tt} \mid \text{ff} \mid F_1 \wedge F_2 \mid F_1 \vee F_2 \mid \langle\alpha\rangle F \mid [\alpha]F$$

4.2 Semantics

- **Diamond ($\langle\alpha\rangle F$):** (Possibility / "Exists") It is *possible* to perform action α and end up in *at least one* state satisfying F.

$$P \models \langle\alpha\rangle F \iff \exists P'.(P \xrightarrow{\alpha} P' \wedge P' \models F)$$

- **Box ($[\alpha]F$):** (Necessity / "For all") *After every* action α, *all* resulting states must satisfy F.

$$P \models [\alpha]F \iff \forall P'.(P \xrightarrow{\alpha} P' \implies P' \models F)$$

  Note: If the process cannot perform α, $[\alpha]F$ is vacuously true.

  Duality: $[\alpha]F \equiv \neg\langle\alpha\rangle\neg F$.

4.3 Distinguishing Formulas (Example)

Let's revisit $P = a.(b + c)$ and $Q = a.b + a.c$. We want a formula F such that $P \models F$ and $Q \not\models F$.

The formula is: $[a](\langle b\rangle\mathbf{tt} \wedge \langle c\rangle\mathbf{tt})$ *Meaning:* "After every action a, I must be able to perform both b and c".

- **For P:** After the unique transition a, we arrive at $(b + c)$. This state can do b (true) and can do c (true). Thus P satisfies the formula.
- **For Q:**
  - If we take the first a, we arrive at $b.\mathbf{nil}$. This state satisfies $\langle b\rangle\text{tt}$ but **not** $\langle c\rangle\text{tt}$.
  - Since Box $[a]$ requires the property to hold for *all* states reachable by a, and one fails, Q does not satisfy the formula.

4.4 Hennessy-Milner Theorem

For finitely branching processes:

$$P \sim Q \iff (P \models F \iff Q \models F, \forall F \in \text{HML})$$

Two processes are bisimilar if and only if they satisfy the same logical HML formulas. If two processes are not bisimilar, there always exists an HML formula that distinguishes them.

Real Languages

This chapter bridges theoretical models (HOFL, CCS) with real-world programming languages, analyzing how abstract concepts translate into concrete implementations.

---

1. Haskell: Pure Functional Programming

Haskell is the practical counterpart of **HOFL**. It is a **purely functional** language (no side-effects, referential transparency) and **lazy** (lazy evaluation).

1.1 Syntax and Functions

Unlike HOFL, Haskell offers advanced syntactic constructs for defining functions.

- **Pattern Matching:**

```
fac 0 = 1
fac n = n * fac (n-1)
```

- **Guards (`|`):** Alternative to nested `if-then-else`.

```
max a b
  | a > b     = a
  | otherwise = b
```

- **Partial Application:** Every function is curried by default.

```
add :: Int -> Int -> Int
add x y = x + y

inc = add 1 -- inc :: Int -> Int (Partial Application)
```

1.2 Data Types and Type Classes

Haskell extends HOFL's type system with user-defined types and overloading.

- Defining New Data Types (data):

  Similar to sum/product types, but named.

```
-- A recursive parametric 'Tree' type 'a'
data Tree a = Leaf a | Node (Tree a) (Tree a)
```

- Type Classes (class, instance):

  Mechanism for overloading (ad-hoc polymorphism). Does not exist in HOFL.

```
-- Class definition (interface)
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool

-- Instance for the Bool type
instance Eq Bool where
    True == True = True
    False == False = True
    _ == _ = False
```

1.3 Lazy Evaluation and Lifted Domains

The defining feature of Haskell is that function arguments are not evaluated until they are strictly necessary.

Theory (HOFL):

This corresponds to the use of **Lifted Domains** ($D_\perp$) in denotational semantics.

- The application rule is $[\![t_1 t_0]\!]\rho = \text{let } \varphi \Leftarrow [\![t_1]\!]\rho.\ \varphi([\![t_0]\!]\rho)$.
- The argument $[\![t_0]\!]\rho$ is passed to the function *without being de-lifted* (without checking if it is $\perp$).

Practice (Haskell):

This allows defining **infinite data structures** (Streams).

```
-- Infinite list of natural numbers
nats = [0 .. ]

-- Fibonacci: defined recursively on the list itself
-- zip and tail work because the list does not need to be fully in memory
fib :: Num b => [b]
fib = 1 : 1 : [ x+y | (x,y) <- zip fib (tail fib) ]
```

> **Comparison:** In an *Eager* (Call-by-Value) language, the definition of `fib` above would cause an immediate infinite loop (stack overflow) by trying to evaluate the argument of `zip` before calling the function.

1.4 Solved Exercises

1. Prime Numbers (Infinite Sieve) Uses laziness to filter an infinite list.

```
factors n = filter (\d -> n `mod` d == 0) [1..n]
prime n = factors n == [1, n]
primes = filter prime [2..] -- Infinite list of primes
```

---

2. Erlang: Actor Model and Distributed Concurrency

Erlang implements the **Actor** model, where each process is an isolated entity that shares no memory and communicates only via asynchronous messages.

2.1 Syntax and Primitives

- **Spawn:** `Pid = spawn(fun)`. Creates a new lightweight process and returns its PID.
- **Send (!):** `Pid ! Msg`. Sends a message **asynchronously**. The sender never blocks (non-blocking).
- Receive with Timeout:

  Extracts messages from the mailbox (infinite local FIFO queue). The after clause handles the timeout.

  ```
  receive
      {From, Msg} ->
          From ! {self(), ok};
      Stop ->
          true
  after 1000 ->
      io:format("Timeout!~n")
  end.
  ```

2.2 Example: Counter Server (CCS Recursion)

In Erlang, mutable state is managed via **tail recursion**, exactly like in CCS (`rec X. P`).

```
-module(counter).
-export([start/0, loop/1]).

start() -> spawn(counter, loop, [0]).

loop(Val) ->
    receive
        inc -> loop(Val + 1);  % Tail call (updated state)
        {From, val} ->
            From ! {self(), Val},
            loop(Val);
        stop -> true
    end.
```

CCS Mapping: $C(n) \stackrel{\text{def}}{=} \text{inc}.C(n+1) + \text{val}.\overline{v_n}.C(n)$.

---

3. Go: Goroutines and Channels (CSP)

Go is inspired by **CSP** (Communicating Sequential Processes).

3.1 Channels and Directionality

Channels are typed and can be unidirectional for safety.

- `chan T`: Bidirectional (read/write).
- `chan<- T`: **Send-only** (write only).
- `<-chan T`: **Receive-only** (read only).

```
// Pipeline example
func producer(out chan<- int) {
    out <- 1
    close(out)
}

func consumer(in <-chan int) {
    msg := <-in
    println(msg)
}
```

3.2 Select (Non-Deterministic Choice)

The `select` construct allows a goroutine to wait on multiple channels simultaneously. Corresponds to the sum + of CCS, but with input/output guards.

- If multiple cases are ready, one is chosen **pseudo-randomly**.
- If no case is ready, it blocks (or executes `default` if present).

```
select {
case msg1 := <-c1:
    println("Received", msg1)
case c2 <- 42:
```

```
    println("Sent 42")
default:
    println("No communication ready (non-blocking)")
}
```

3.3 Channels: Unbuffered vs Buffered

Type	Syntax	Corresponding CCS Semantics
Unbuffered	`make(chan T)`	**Synchronous** (Handshake). $\alpha.\mathbf{nil} \mid \bar{\alpha}.\mathbf{nil} \xrightarrow{\tau} \mathbf{nil}$. Sender and receiver must synchronize (Rendezvous).
Buffered	`make(chan T, N)`	**Asynchronous** (up to N). The sender does not block until the buffer is full.

3.4 Example: Deadlock in Go (vs CCS)

A deadlock in Go on unbuffered channels is identical to a CCS process blocked on an action without a partner.

```
package main
func main() {
    ch := make(chan int) // Synchronous
    ch <- 42 // DEADLOCK! No one is listening.
    // In CCS: 'a.nil (without the partner in parallel)
}
```

Advanced Concurrency

This chapter extends the CCS theory to handle abstraction (ignoring internal τ actions) and introduces temporal logics for the formal verification of properties.

---

1. Weak Bisimulation

Strong bisimulation ($\sim$) is often too restrictive: it distinguishes processes that have the same observable behavior but differ in internal τ (tau) actions. Example: A buffer that performs an internal τ step to move data ($a.\tau.\bar{b}$) should be equivalent to an ideal buffer ($a.\bar{b}$), but $\sim$ distinguishes them.

1.1 Weak Transition ($\Longrightarrow$)

We define a new transition relation that "skips" τ actions.

- **Silent Weak Transition ($\overset{\tau}{\Longrightarrow}$):** A sequence of zero or more τ steps.

$$p \overset{\tau}{\Longrightarrow} q \iff p(\xrightarrow{\tau})^* q$$

(Note: $p \overset{\tau}{\Longrightarrow} p$ is always true).

- **Observable Weak Transition ($\overset{\alpha}{\Longrightarrow}$):** An action α "surrounded" by τ steps.

$$p \overset{\alpha}{\Longrightarrow} q \iff \exists p', q'.\ p \overset{\tau}{\Longrightarrow} p' \overset{\alpha}{\to} q' \overset{\tau}{\Longrightarrow} q$$

Meaning: do some internal work, then the visible action, then more internal work.

1.2 Definition of Weak Bisimulation ($\approx$)

A relation $\mathcal{R}$ is a weak bisimulation if, for every $(p, q) \in \mathcal{R}$:

1. If $p \overset{\alpha}{\to} p'$, then $\exists q'$ such that $q \overset{\alpha}{\Longrightarrow} q'$ and $(p', q') \in \mathcal{R}$.
2. If $p \overset{\tau}{\to} p'$, then $\exists q'$ such that $q \overset{\tau}{\Longrightarrow} q'$ and $(p', q') \in \mathcal{R}$. *(Note: here q can respond by staying still, since $\overset{\tau}{\Longrightarrow}$ includes zero steps).*

Two processes are **Weakly Bisimilar** ($p \approx q$) if there exists a weak bisimulation containing them.

1.3 The Congruence Problem (Summation)

Weak bisimulation is **NOT a congruence** with respect to the sum operator ($+$).

Counterexample: Let $P = a.\mathbf{nil}$ and $Q = \tau.a.\mathbf{nil}$. Clearly $P \approx Q$ (the initial τ is weakly unobservable).

Let us place them in a sum context: $C[\cdot] = \cdot + b.\mathbf{nil}$.

- $C[P] = a.\mathbf{nil} + b.\mathbf{nil}$
- $C[Q] = \tau.a.\mathbf{nil} + b.\mathbf{nil}$

$C[Q]$ can perform τ and become $a.\mathbf{nil}$ (discarding the option b). $C[P]$ cannot simulate this τ step by becoming a state that has lost option b (if it stays still, it still has b; if it performs a or b, it has performed a visible action). Thus $C[P] \not\approx C[Q]$.

1.4 Observational Congruence

To solve the problem, we define observational equality (or weak congruence): Two processes p, q are congruent if:

1. $p \approx q$
2. For every α (including τ), if $p \overset{\alpha}{\to} p'$ then $q \overset{\alpha}{\Longrightarrow} q'$ (with at least one τ step if $\alpha = \tau$).

---

2. Temporal Logics (LTL & CTL)

Temporal logics allow specifying properties such as "Safety" (nothing bad ever happens) and "Liveness" (something good eventually happens).

2.1 LTL (Linear Temporal Logic)

Interprets time as a straight line (a single execution trace). Formulas are evaluated over a path $\pi = s_0 \to s_1 \to s_2 \ldots$

Operators:

- **X (Next):** $\pi \models \mathbf{X}\phi$ if ϕ holds in s_1.
- **G (Globally):** $\pi \models \mathbf{G}\phi$ if ϕ holds in all s_i (Safety).
- **F (Finally):** $\pi \models \mathbf{F}\phi$ if $\exists i$ s.t. ϕ holds in s_i (Liveness).
- **U (Until):** $\pi \models \phi\mathbf{U}\psi$ if ψ holds eventually, and ϕ holds until that moment.

2.2 CTL (Computation Tree Logic)

Interprets time as a tree (Branching Time). In each state, multiple futures are possible. Every temporal operator must be preceded by a path quantifier.

Quantifiers:

- **A (All):** For all future paths.
- **E (Exists):** There exists at least one future path.

CTL Examples:

- **AG**(safe): In all paths, in all states, I am safe (Strong Invariant).
- **EF**(start): There exists a path where eventually I start (Reachability).
- **AF**(end): In all paths, eventually I finish (Guaranteed Termination).

2.3 Expressiveness Comparison

- **LTL: FG**p (On this path, eventually p will be continuously true).
- **CTL: AGEF**p (It is always possible to reset the system and reach p).
- They are incomparable: there are LTL properties not expressible in CTL and vice versa.

---

3. The μ-Calculus

It is the most powerful logic (subsumes LTL and CTL). It is based on fixpoints.

3.1 Basic Syntax

$$\phi ::= \mathrm{tt} \mid \mathrm{ff} \mid Z \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \langle\alpha\rangle\phi \mid [\alpha]\phi \mid \mu Z.\phi \mid \nu Z.\phi$$

3.2 Fixpoints and Properties

- $\mu Z.\phi$ **(Least Fixed Point):** Used for **Liveness** / Reachability.
  - Example: $\mu Z.(p \vee \langle\cdot\rangle Z) \rightarrow$ "Either p holds now, or I can take a step and retry". (Corresponds to **EF**p).
  - *Intuition:* Looks for a "finite proof" (a path ending in p).
- $\nu Z.\phi$ **(Greatest Fixed Point):** Used for **Safety** / Invariants.
  - Example: $\nu Z.(p \wedge [\cdot]Z) \rightarrow$ " p holds now AND after every step Z still holds". (Corresponds to **AG**p).
  - *Intuition:* Allows infinite loops where p is always true.